Response to Reviewer 3

Enhancing Consent Policy Privacy in Blockchain-Based Healthcare Data Sharing

---

Working draft. Page and line references will be added after the revised manuscript reaches final pagination.

Comments 1: *The trust model around the Trusted Proof Service is a major limitation and should be discussed more critically. The paper explicitly assumes that the TPS preserves policy material and requester evidence, but the current evaluation even co-locates the TPS with the provider. This weakens the practical privacy claim, since the most sensitive information is moved off-chain rather than cryptographically protected from the service itself. The paper should more clearly distinguish what is protected from public-chain observers and what is merely trusted to the TPS.*

Response 1: We agree that this is the main architectural limitation, and the revision states it as such in the Problem, Discussion, and Conclusion sections. The distinction the reviewer asks for is now made in two places. The trust-model subsection separates authorization correctness, which the commitment and the on-chain verification enforce regardless of the TPS (and which the revised relation strengthens by binding the program length, the authorization context, and the issuer-signed claims), from policy confidentiality, which is not cryptographically enforced against the TPS and rests on the service following its workflow; it also names what a compromised or colluding TPS can do (expose policy and evidence, substitute unsigned evidence, refuse to prove) and notes that the provider-operated TPS of the evaluation concentrates that trust at the provider. A new threat/guarantee table then lists, for the public-chain observer, the requester, an honest-but-curious TPS, a compromised or colluding TPS, the provider-operated TPS, and a forged issuer, which of policy confidentiality, authorization correctness, liveness, and accountability is cryptographic, TPS-dependent, external, or not provided.

The claim is scoped accordingly: what the evaluation establishes is what a public-chain observer can infer from the artifacts a workflow-following TPS produces, and the abstract, Discussion, and Conclusion say that confidentiality against the service itself is a trust assumption, not a cryptographic property. The Discussion also names the mechanisms that would relocate that trust—collaborative-SNARK or attested proof generation so that no single party sees both policy and evidence, a holder-signed authorization context so that the service cannot reuse evidence it has seen, and accountable proof obligations so that refusals become attributable—and the Related Work section now relates this to self-sovereign-identity designs in which claim release is holder-controlled (Comment 5).

Location in the revised manuscript: Section “Problem and Threat Model,” subsection “System and Trust Model,” and the new threat/guarantee table; Section “Discussion”; Section “Conclusion,” third paragraph.

> Policy confidentiality against the TPS itself is not cryptographic but rests on the service following its workflow, and the evaluation’s provider-operated TPS concentrates that trust at the provider.

Comments 2: *The current proof statement leaves important soundness gaps. The paper acknowledges that the program length is not bound into the policy commitment and that the proof*

is not bound to the requester, resource, or session context. These are not minor implementation details: they directly affect the correctness and replay-resistance of the authorization workflow. These issues should be fixed in the design and experimentally re-evaluated, rather than mainly left to future work.

Response 2: Both gaps are closed in the revised design and implementation, and the evaluation was repeated on the revised system.

The policy commitment now covers the program length, $h = \mathsf{Poseidon}(\mathsf{H}_{\mathrm{pol}}(P_{\mathrm{enc}}) \parallel \ell \parallel r)$, and the same private ℓ drives both the commitment check and the interpreter, so an authorizing proof cannot rest on a shorter stack-consistent prefix. The proof statement now carries a public context digest $d_\chi = \mathsf{Poseidon}(id_{\mathrm{req}} \parallel id_{\mathrm{res}} \parallel \nu)$ that the circuit constrains and the access-control contract recomputes from the transaction parameters, so a captured proof cannot be paired with a fresh nonce, another resource, or another requester within a deployment. The measured marginal costs (88 constraints for the length binding; 605 constraints, one public input, and about 56,000 gas for the context binding including the on-chain Poseidon recomputation) are given in the responses to Reviewer 1, Comments 1 and 2, which also describe the regression and end-to-end tests.

The re-evaluation is complete rather than partial. All of-record numbers in the revised manuscript come from a new capture of 1,077 policies and 1,773 Polygon Amoy executions under the frozen revised release (five-signal statement, length- and context-bound relation, credential-derived profile attributes), followed by a new un-padded ablation run of 1,119 executions on the same policy subset. The cost profile reflects the revised circuit (81,740 constraints) and the revised on-chain path, and the results sections report the outcome of the privacy attacks on the revised transcript, including two criterion failures that we report and trace to the padding's random draws rather than to the policy (Reviewer 1, Comments 5 and 6).

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Commitment Binding and Proof Generation"; Section "Privacy Evaluation" (all result tables and the cost profile).

Comments 3: *The policy language is still quite limited. The system focuses on fixed-length Boolean RPN programs, while many realistic healthcare consent policies involve temporal constraints, revocation, obligations, delegation, purpose limitation, institutional approval, and evolving consent. Some of these are treated as external predicate bits, but this moves complexity outside the proven statement. The paper should better explain what kinds of real consent policies can actually be supported end-to-end.*

Response 3: We agree that "external predicate bit" left the guarantee unclear, and the revision answers the question in two steps: the relation no longer accepts unconstrained predicate bits, and the Design section now states feature by feature what the prototype establishes at authorization time.

In the revised relation every predicate bit is derived in-circuit from a witness bound to committed parameters: ontology-scoped conditions (disease, study design, ancestry, geography, clinical term) are Merkle-membership proofs against a provider-committed whitelist root; user-, project-, and institution-specific restrictions are membership proofs of the holder pseudonym in provider-committed allowlists; and the profile-backed data-use terms are derived from the claims of an EdDSA-Poseidon credential that binds the policy commitment, the requester pseudonym, the resource, the claims, a validity interval, and a credential identifier. The policy's own validity window is committed and checked against the public request epoch, which the contract requires to equal the current hour.

A new table in the Design section classifies each consent feature as decided in-circuit, decided on-chain by the access-control contract from public state, committed but not decided at request time, or not provided. The Conclusion lists the unsupported features among its limitations, and the Discussion suggests future work to extend the class further (a governed issuer registry with in-circuit membership and revocation witnesses, and holder-controlled release of claims) without changing the interpreter or the public interface, because new conditions enter through predicate slots and the fixed-format parameter commitment.

Location in the revised manuscript: Section "TPS-ZKPI Design," subsection "Policy Representation and Interpretation," new paragraph "Scope of the supported policies" and the new policy-support table; Section "Conclusion," third paragraph.

> The policy class is deliberately restricted, and the policy-support table states what the RC1 prototype establishes end-to-end. Categorical data-use terms, ontology-scoped conditions, requester allowlists, profile-backed terms, the policy validity window, and time-limited use are decided at authorization time, either inside the proved relation or by the ACM from public state. A publication moratorium is bound into the policy as an accepted obligation but cannot be decided at request time, negation is rejected by the DUO compiler because membership witnesses prove inclusion only, and purpose limitation, post-access duties, delegation, and credential revocation are left to the data access agreement and the issuer lifecycle. The prototype is therefore an access-time consent checker for bounded Boolean policies over standardized terms, not a general consent-language engine.

Comments 4: *The privacy evaluation is too narrow. The adversary is mainly a passive metadata-only public-chain observer, and the effective leakage channel is reduced mostly to gas usage. This is useful, but it does not cover adaptive requester probing, repeated access attempts, timing/order leakage, off-chain TPS communication patterns, compromised TPS behaviour, or correlations between authorization outcomes and policy classes. The paper should either broaden the evaluation or tone down the privacy claims.*

Response 4: We chose to tone down the claims and to say precisely what the evaluation covers, while making the evaluation of the modeled channel more rigorous.

Toned-down claims. The abstract, Discussion, and Conclusion now describe the result as empirical resistance against the modeled public-chain metadata attacks, and the Discussion lists every exclusion the reviewer names—timing and ordering, mempool and broadcast behaviour, off-chain TPS communication, adaptive probing, repeated attempts, correlations between outcomes and policy classes (every request in the corpus is granted, so this correlation is not exercised), and compromised TPS behaviour—with the statement that the results should not be read as establishing general policy confidentiality against them. The threat model also states why the gas channel is wider than the verifier's arithmetic: the EVM prices calldata per byte, so any deterministic difference in how a public argument is encoded reaches the observer through gas even when calldata length is constant.

A more rigorous treatment of the modeled channel. The evaluation was re-run on the revised system and a new corpus. Two of the 45 supervised tasks now meet the failure criterion; we report them as failures and trace them, by subtracting the logged pad target from the access gas, to coincidences of the padding's own random draws with a label (Reviewer 1, Comments 5 and 6), and we quantify the criterion's false-positive rate with a null simulation. The re-run ablation shows what the un-padded transcript leaks and why: not the policy logic, which the fixed circuit already hides, but the byte pricing of the requester pseudonym, whose encoding

width in our harness correlates with whether a policy activates the profile predicate. We report this because it is the kind of channel the reviewer's broader adversary would exploit, and because it shows that the evaluation does detect sub-kilogas deterministic channels when they exist.

We did not add adaptive or timing experiments in this revision. They require a different corpus design (repeated, adversarially chosen requests against the same policy, and control over block inclusion) and an adversary model with an active component; the Conclusion names them as the next step for the evaluation.

Location in the revised manuscript: Abstract; Section "Privacy Evaluation," subsection "Adversary and observables"; Section "Discussion," first two paragraphs; Section "Conclusion," third and fourth paragraphs.

> That channel is wider than the verifier's arithmetic: because the EVM prices calldata per byte (four gas for a zero byte, sixteen otherwise), the gas of an access transaction also encodes the byte pattern of its public arguments, so any deterministic difference in how an argument is encoded reaches the observer through gas even when the calldata length is constant.

Comments 5: *The comparison with related work should be strengthened by considering recent blockchain-based access-control architectures that combine smart contracts, SSI-oriented trust models, and cryptographic mechanisms for controlling access to sensitive claims. In particular, the paper should better position TPS-ZKPI with respect to the issues discussed in DOI: 10.1016/j.bcra.2024.100196, especially regarding verifier authorization, smart-contract accountability, off-chain trust assumptions, and practical access-control workflows.*

Response 5: We thank the reviewer for the pointer. The Related Work section now contains a paragraph on self-sovereign-identity architectures that positions TPS-ZKPI against Buccafurri, De Angelis, and Nardone (Blockchain: Research and Applications, 2024) along the four dimensions named, together with recent zero-knowledge authentication work (Madine et al., 2025) and surveys of zero-knowledge proofs for blockchain privacy.

- *Verifier authorization.* In the CP-ABPRE design the holder's claims are readable only by verifiers that satisfy a holder-defined policy. In TPS-ZKPI the protected object is the provider's consent policy, not the requester's claim set: the claims are consumed as private witnesses of a proof whose statement is fixed and whose verifier is a public contract, so any observer may verify and verifier authorization is not a design question.
- *Smart-contract accountability.* There the contract meters the release of claims and keeps verifiers accountable; here the contract records the context-bound authorization outcome, and accountability attaches to that record rather than to claim release.
- *Off-chain trust.* An SSI verifier that decrypts a claim learns it; the TPS learns both policy and claims. This is why the Problem section treats the TPS as an explicit trust boundary and the new threat/guarantee table states what survives its compromise (Comment 1).
- *Access-control workflow.* The two designs sit on opposite sides of the same authorization relation, and we note that combining them—holder-controlled release of exactly the claims that TPS-ZKPI consumes as witnesses—would remove the TPS's view of the requester evidence; the Discussion lists this among the reduced-trust directions.

Location in the revised manuscript: Section "Related Work," new paragraph on self-sovereign-identity architectures; reference list.

> TPS-ZKPI sits on the other side of the same authorization relation. In SSI designs the protected object is the requester's claim set and the policy that governs it is the holder's, whereas here the protected object is the provider's consent policy: the requester's claims are consumed as private witnesses of a proof whose statement is fixed and whose verifier is a public contract, so verifier authorization is not a design question (any observer may verify) and accountability is carried by the recorded, context-bound outcome rather than by a contract that meters the release of claims.

Comments 6: *The cost and scalability analysis needs more depth. The reported gas-padding overhead is substantial, around 38% for access and 40% for commit operations, and the evaluation is performed at a single circuit depth. The paper should evaluate how costs change with larger policy programs, richer predicates, more complex DUO/ICD conditions, and more realistic access workloads. Otherwise, it is difficult to judge whether the design is practical for real healthcare data-sharing platforms.*

Response 6: We added a cost-scaling analysis and restructured the cost discussion around the distinction that decides practicality: which costs are paid on-chain, and which off-chain.

Larger policy programs. A new table reports the prover-side cost of the revised circuit at program lengths 32, 64 (the deployed value), and 128 with the attribute and predicate counts fixed: R1CS constraints grow from 64,945 to 81,740 to 134,009, the proving key from 39 to 52 to 88 MB, and proving time from 0.38 to 0.49 to 0.73 s per proof on a laptop-class prover (witness generation 0.28–0.54 s). The verification side does not change: the statement keeps its five public signals at every length, so the verifier contract, its gas, and the padding band are the same.

Richer predicates and more complex DUO/ICD conditions. These enter through the predicate slots and their gadgets (Merkle membership for ontology scopes and allowlists, the credential verifier for profile-backed terms) rather than through the program length. Each additional slot adds its gadget's constraints to the prover and nothing to the on-chain cost, because predicate parameters are bound through the fixed-format parameter commitment rather than exposed as public inputs; a larger ICD or OBI whitelist changes only the Merkle root the provider commits, not the circuit or the gas. The revised corpus exercises this: whitelists range from 2 to 32 codes across five vocabularies, and the access gas of the un-padded ablation run varies by 276 gas across 1,119 executions.

Access workloads. The padding overhead is a fixed premium per transaction (about 330,000 gas on access and 85,000 on commit; 0.032 versus 0.022 POL per access at the evaluation's gas price), so it scales linearly with the number of authorizations and does not compound; the Discussion now states this together with how the band is calibrated per network (Reviewer 1, Comment 7). We did not run a throughput benchmark: the on-chain path is one Groth16 verification plus bookkeeping per authorization and is bounded by the chain, not by the design, while the off-chain path is embarrassingly parallel across TPS instances.

Location in the revised manuscript: Section "Privacy Evaluation," subsection "Ablation," paragraph "Cost" and the new cost-scaling table; Section "Discussion," paragraph "The cost of padding."

> The cost-scaling table shows how the prover-side cost scales when the program length grows from 32 to 128 with N and M fixed: constraints, proving-key size, and proving time grow roughly linearly with the program length (from 65,000 to 134,000 constraints and from 0.38 to 0.73 s per proof on a laptop-class prover), while the statement keeps its five public signals, so the verifier contract, its gas, and the

padding band are unchanged. Richer predicates would enter through M and the per-slot gadgets rather than through the program length; each additional slot adds its gadget's constraints to the prover and nothing to the on-chain cost, since predicate parameters are bound through the fixed-format parameter commitment rather than exposed as public inputs.